When the cluster starts, you need to initialize both projections: `BetProjection` and `MarketProjection`. This is done with an `init` method that uses the `ShardedDaemonProcess` to spawn as many actors/projections as tags in the entities, bet and market. Each projection is wrapped in a `Behavior` distributed in a sharded fashion across the cluster. Here is the `MarketProjection`:

```
object MarketProjection {

  def init(system: ActorSystem[_]): Unit = {
    val producer = createProducer(system)
    val topic =
      system.settings.config.getString("kafka.market-projection.topic")    ❶

    ShardedDaemonProcess(system).init(
      name = "MarketProjection",
      Market.tags.size,
      index =>
        ProjectionBehavior(                                                 ❷
          createProjection(system, topic, producer, index)),
      ShardedDaemonProcessSettings(system),                                 ❸
      Some(ProjectionBehavior.Stop))                                        ❹
  }
  ...
}
```

❶ Reads the topic name from the configuration

❷ Wraps up the projection in a ProjectionBehavior

❸ The projection behavior is sharded in the cluster.

❹ Message sent to the behavior to stop it in case of a rebalance or shutdown

The initialization of both projections is almost identical, as you can see in the bet projection:

```
object BetProjection {
  ...

  def init(
      system: ActorSystem[_],
```